Frequency Assignment

Advanced Artificial Intelligence

Submitted by :

Name: Khalil Gharssallah

Student ID: 2430150037

Under the Supervision of

ÖZLEM FEYZA ERKAN

Department of Computer Engineering

Beykoz University

Academic Year: 2025/2026

Contents

List of Acronyms **vi**

Abstract **vii**

1 Introduction **1**

1.1 Context and Motivation 1

1.2 Artificial Intelligence Perspective 1

1.3 Implementation-Oriented Objective 2

1.4 Organization of the Report 2

2 Problem Formulation **3**

2.1 Problem Definition 3

2.2 Formal Mathematical Formulation 3

2.2.1 Hard Constraints 3

2.2.2 Conflict Function 3

2.3 Graph-Theoretic Representation 4

2.4 Problem Characteristics and Solution Paradigms 4

2.5 Mapping the Model to Code 4

3 CSP Backtracking Approach **5**

3.1 Principle of Backtracking 5

3.2 Algorithm Description 5

3.3 CSP Concepts vs Implementation 5

3.4 Behavior and Limitations 6

3.5 Summary 6

4 Local Search Approach **7**

4.1 Principle of Local Search 7

4.2 Conflict Minimization Strategy 7

4.3 Algorithm Description 8

4.4 Mapping to the Python Implementation 8

4.5 Behavior and Characteristics 8

4.6 Advantages and Limitations . . . 9
4.7 Summary . . . 9

5 Experimental Setup **10**
5.1 Experimental Objectives . . . 10
5.2 Problem Instance Generation . . . 10
5.3 Experimental Parameters . . . 11
5.4 Performance Metrics . . . 11
5.4.1 Execution Time . . . 11
5.4.2 Solution Quality . . . 11
5.4.3 Robustness . . . 11
5.5 Experimental Procedure . . . 12
5.6 Implementation Environment . . . 12
5.7 Summary . . . 12

6 Results and Visualization Analysis **13**
6.1 Introduction . . . 13
6.2 Experimental Environment . . . 13
6.3 Interference Graph Visualization . . . 14
6.4 Seeded Layout and Reproducibility . . . 14
6.5 Execution Time Analysis . . . 15
6.6 Conflict Analysis . . . 15
6.7 Impact of Interference Probability . . . 16
6.8 Frequency Assignment Comparison . . . 17
6.9 Role of the Web Interface . . . 17
6.10 Web-Generated Results Report . . . 17
6.10.1 What the Web Report Contains . . . 17
6.10.2 How It Supports the Experimental Analysis . . . 18
6.10.3 Sample Generated Report . . . 18
6.11 Conclusion of Results . . . 24

7 Discussion **25**
7.1 Interpretation of Experimental Results . . . 25
7.2 Exact Versus Heuristic Trade-off . . . 25
7.3 Impact of Problem Constraints . . . 26
7.4 Impact of Visualization and User Interaction . . . 26
7.5 Reproducibility and Experimental Control . . . 26
7.6 Development and Implementation Considerations . . . 27
7.7 Limitations of the Study . . . 27
7.8 Summary . . . 27

8 Conclusion and Future Work **28**
8.1 Conclusion 28
8.2 Future Work 28
8.3 Final Remarks 29

List of Figures

6.1 Interference graph rendered by the web interface 14
6.2 Average execution time comparison 15
6.3 Conflict comparison between CSP and Local Search 16

List of Tables

2.1 Mapping Between Formal Model and Python Implementation 4

3.1 CSP Concepts and Their Implementation 5

4.1 Mapping Between Local Search Concepts and Python Implementation . . 8

5.1 Experimental Parameters 11

6.1 Effect of interference probability on solution quality 16

6.2 Frequency assignments produced by both algorithms 17

List of Acronyms

FAP	Frequency Assignment Problem
AI	Artificial Intelligence
CSP	Constraint Satisfaction Problem
DFS	Depth-First Search
LS	Local Search
RF	Radio Frequency
QoS	Quality of Service
FAP	Frequency Assignment Problem
NUMSTATIONS	`NUM_STATIONS` – Number of Base Stations
NUMFREQUENCIES	`NUM_FREQUENCIES` – Number of Available Frequencies
INTERFERENCEPROB	`INTERFERENCE_PROB` – Interference Probability
REPEATS	`REPEATS` – Number of Experimental Repetitions

Abstract

The Frequency Assignment Problem (FAP) is a central problem in wireless networks. It involves assigning a finite number of frequency channels to an expanding set of base stations. The goal is to do this without causing interference. With growing network density and limited spectral resources, this problem is very constrained and hard to compute.

This project models the Frequency Assignment Problem. We use a graph-based artificial intelligence approach, where base stations act as nodes, and their interference relations are edges. The issue is formally defined by decision variables, hard constraints, and a conflict minimization objective, making it suitable for various AI paradigms.

Different solution methods were put into practice and then compared.

One approach treated the problem like a Constraint Satisfaction Problem. This involved using a backtracking algorithm that was made better with Dynamic Variable Ordering. This helped in searching for solutions more efficiently and finding problem areas sooner.

The other approach used a Local Search method. It was based on a Value Jumping heuristic. This method steadily got better at assigning values by directly picking frequency values that brought down the total number of conflicts.

We evaluated both approaches under varying problem conditions. We used automatically generated interference graphs that had different sizes and constraint densities. The performance of each approach was compared based on computational efficiency, solution quality, scalability, and how they behaved in over-constrained scenarios. Experimental results indicate that while the CSP approach consistently guarantees conflict-free solutions when such solutions exist, it faces scalability limitations as problem complexity grows. The Local Search method, conversely, offers quicker and more scalable solutions, but this comes at the expense of allowing a small number of residual conflicts to remain.

The project includes a web interface for better understanding and replicating results. This interface lets users adjust parameters interactively, view interference graphs graphically, and compare different algorithm outputs directly. It also creates short experimental reports automatically. The findings show a basic trade-off between precise and approximate methods. This confirms the value of visualization when studying large-scale frequency assignment problems.

Chapter 1

Introduction

1.1 Context and Motivation

Wireless communication networks depend on many base stations that share a limited radio spectrum. Close base stations can't use the same frequency because of interference. When frequency allocation is poor, it can cause signal problems, reduce service quality, and make the network run poorly.(Aardal and Hurkens, 2002).

In this situation, the frequency assignment problem comes up. It involves giving frequencies to base stations so that interference rules are followed. As the number of base stations goes up, the possible assignments increase a lot, which makes it hard to check every option.(Russell and Norvig, 2021).This encourages the use of AI methods that can manage big search areas and hard rules.

1.2 Artificial Intelligence Perspective

When tackling the frequency assignment problem using Artificial Intelligence, it's viewed as a standard constraint-based optimization challenge. Two chief types of AI approaches are applicable. One involves exact methodologies, like Constraint Satisfaction Problems, where the goal is to locate solutions that fully meet all requirements. The other uses heuristic techniques, such as Local Search, which loosen the requirements to instead lower the amount of violations. (Dechter, 2003; Minton et al., 1992).

This project examines both methods by coding them directly in Python. Applying each to the same problem allows for a direct comparison of their speed, the quality of the result, and how well they perform.

1.3 Implementation-Oriented Objective

This project aims to link theoretical AI concepts with practical application via code implementation. Python data structures model base stations, frequencies, and interference. We implement a constraint satisfaction problem backtracking algorithm and a local search heuristic. Experimental evaluation focuses on practical insights instead of just theoretical analysis.

1.4 Organization of the Report

This report is structured in the following way: Chapter 2 gives the problem formulation and shows how it relates to the code. Chapter 3 describes the CSP backtracking method. Later chapters cover the Local Search method, the experimental setup, and a comparison of the results.

Chapter 2

Problem Formulation

2.1 Problem Definition

The frequency assignment problem involves assigning a unique frequency to each base station, ensuring that nearby stations don't use the same one to avoid interference. With a limited number of frequencies available, this task can be tricky and may not always allow for a solution without any conflicts. (Aardal and Hurkens, 2002).

2.2 Formal Mathematical Formulation

For each base station $v \in V$, a decision variable is defined:

$$x_v \in F$$

where $F = \{1, 2, \ldots, K\}$ is the set of available frequencies.

2.2.1 Hard Constraints

For every interference edge $(u, v) \in E$, the following constraint must hold:

$$x_u \neq x_v$$

2.2.2 Conflict Function

When the problem is over-constrained, conflicts are measured as:

$$\text{conflicts}(x) = \sum_{(u,v) \in E} \mathbb{I}[x_u = x_v]$$

where $\mathbb{I}[\cdot]$ is an indicator function.

2.3 Graph-Theoretic Representation

The frequency assignment problem can be represented as an undirected graph $G = (V, E)$, where:

- V represents the set of base stations.
- E represents interference relationships between stations.

An edge $(u, v) \in E$ indicates that stations u and v are geographically close and therefore cannot be assigned the same frequency.

2.4 Problem Characteristics and Solution Paradigms

The inherent structure of the problem lends itself to two distinct Artificial Intelligence approaches :

- **Constraint Satisfaction Problem (CSP):** The problem is addressed by identifying an assignment that meets all strict constraints. This method is appropriate when a viable solution is present.
- **Local Search Optimization:** When a problem has too many limits, the goal shifts to reducing the number of issues, not getting rid of them all.

The dual nature of the Frequency Assignment Problem makes it particularly suitable for comparing backtracking-based CSP techniques and heuristic local search strategies.

2.5 Mapping the Model to Code

Table 2.1: Mapping Between Formal Model and Python Implementation

Formal Element	**Python Implementation**
Base stations V	`stations = [f"S{i}"]`
Frequencies F	`frequencies = list(range(...))`
Interference edges E	`graph.add_interference()`
Constraint $x_u \neq x_v$	Neighbor consistency check
Conflicts function	`count_total_conflicts()`

Explanation: The Python code closely follows the math, so the theory and the actual implementation match up.

Chapter 3

CSP Backtracking Approach

3.1 Principle of Backtracking

Backtracking, a depth-first search method, is often applied to tackle Constraint Satisfaction Problems. The approach develops assignments step by step, stopping when it finds a constraint is not met. This helps avoid exploring paths that won't work. (Dechter, 2003).

3.2 Algorithm Description

The solution uses a recursive Python function to carry out backtracking. The process involves selecting an unassigned base station and assigning it a frequency at each step. The frequencies of neighboring stations are checked in the interference graph to confirm consistency.

3.3 CSP Concepts vs Implementation

Table 3.1: CSP Concepts and Their Implementation

CSP Concept	**Python Representation**
Variable	Base station identifier
Domain	List of frequencies
Assignment	Python dictionary
Constraint checking	Neighbor comparison
Search strategy	Recursive backtracking

Explanation: The table shows how theoretical CSP elements are turned into actual Python structures, making the implementation clear and easy to learn.

3.4 Behavior and Limitations

The CSP backtracking method ensures a solution without conflicts if one is possible. Yet, the calculation time increases greatly as the number of base stations rises. In areas with many interfering signals or few available frequencies, the algorithm might not find a solution quickly.

3.5 Summary

Constraint Programming backtracking can act as a precise reference point for solving frequency assignment issues. Because it is accurate, it works well for small to mid-sized situations, but its limits push us to use approximation methods like Local Search.

Chapter 4

Local Search Approach

4.1 Principle of Local Search

Local search is a method that relies on trial and error to solve problems. It starts with a possible full solution and then tries to make it better, rather than building a solution from scratch. Unlike precise search methods, local search does not promise a perfect solution. Instead, it seeks to improve the quality of the solution step by step by reducing a target function. This makes local search a good choice for very big or very restricted problems. (Minton et al., 1992).

Local Search, when used for frequency assignment, begins by assigning frequencies to base stations. This initial assignment might have some interference issues. The algorithm then works to fix this by repeatedly changing the assignment to cut down on these conflicts.

4.2 Conflict Minimization Strategy

The local search method in this project uses a conflict-minimization approach. The algorithm checks base stations with interference issues in each step and tries to change their frequencies to lower the total number of conflicts.

Formally, the algorithm aims to minimize the following objective function:

$$\min_{x} \; ; \text{conflicts}(x)$$

where x represents a complete frequency assignment for all base stations, and conflicts(x) denotes the total number of interference conflicts in the assignment, as defined in Chapter 2.

4.3 Algorithm Description

The Local Search algorithm, when coded in Python, generally involves these steps:

1. Start by randomly assigning frequencies to all base stations.
2. Figure out the total number of conflicts with the present assignment.
3. Pick a base station that has at least one conflict.
4. Change its frequency to one that lowers the number of conflicts.
5. Keep repeating this until a stopping point is reached.

This repeating process of getting better bit by bit lets the calculation look at possible answers in a good way without checking every single option.

4.4 Mapping to the Python Implementation

Table 4.1: Mapping Between Local Search Concepts and Python Implementation

Local Search Concept	**Python Implementation**
Initial solution	Random frequency assignment
Objective function	`count_total_conflicts()`
Neighborhood	Frequency changes for one station
Heuristic	Conflict minimization
Stopping condition	Maximum iterations or no improvement

Explanation: The table shows how Local Search ideas appear in Python code. The method works by checking nearby options made when the radio frequency of one base station is changed.

4.5 Behavior and Characteristics

Local Search, different from CSP backtracking, always gives a full assignment, even if the problem has too many restrictions. But, this assignment might still have problems. It scales well and converges fast, especially with many base stations.

The choice of heuristic and when to stop searching are important for Local Search. Even so, its simple nature and efficiency make it useful for real-world frequency assignment cases where meeting all requirements is not possible.

4.6 Advantages and Limitations

Advantages:

- High scalability for large problem instances.
- Fast convergence compared to exact methods.
- Always produces a complete assignment.

Limitations:

- Does not guarantee a conflict-free solution.
- Solution quality depends on heuristic design.
- May converge to local minima.

4.7 Summary

Compared to CSP backtracking, Local Search gives you a practical, heuristic way to tackle the frequency assignment problem. You trade off completeness, but you gain speed and scalability. That makes it a strong choice for big or complicated cases—exactly the kind of situations this project targets. So, including Local Search here isn't just reasonable; it's necessary.

Chapter 5

Experimental Setup

5.1 Experimental Objectives

We set up these experiments to see how CSP Backtracking and Local Search stack up against each other when facing the same problems. We're looking at how fast they run, how good their solutions are, and how well they handle tougher, more constrained situations.

To keep things fair—and make it easy for others to repeat our tests—we use automatically generated problem instances every time.

5.2 Problem Instance Generation

Every experiment starts with a fresh interference graph, generated at random. By setting a fixed random seed, I make sure you always get the same problem instances, no matter how many times you run things.

The following parameters define a problem instance:

- Number of base stations
- Number of available frequencies
- Interference probability between stations

In Python, I create a list of base station IDs, then add interference edges between pairs. The connections aren't just thrown in—each edge appears based on probability, which shapes the graph differently each time, but always in a reproducible way.

5.3 Experimental Parameters

Table 5.1 summarizes the main parameters used in the experiments.

Table 5.1: Experimental Parameters

Parameter	**Value**
Number of stations (`NUM_STATIONS`)	8
Number of frequencies (`NUM_FREQUENCIES`)	3
Interference probability (`INTERFERENCE_PROB`)	0.45
Number of repetitions (`REPEATS`)	50
Random seed	-

Explanation: The table shows the parameters pulled straight from the Python code. These numbers set up problems that aren't too easy or too hard, making it possible to really compare exact methods with heuristic ones.

5.4 Performance Metrics

Three performance metrics are used to evaluate the algorithms:

5.4.1 Execution Time

To measure execution time, I run each algorithm `REPEATS` times. Then I take the average. This smooths out any random spikes or dips, giving a fairer sense of how each approach actually performs.

5.4.2 Solution Quality

To measure solution quality, we count up the conflicts in the final assignment. Basically, if two base stations that interfere with each other end up with the same frequency, that's a conflict. With the CSP approach, you're looking for perfection—zero conflicts means you've nailed it. Local Search works differently; it aims to keep conflicts as low as possible, but doesn't always get rid of them entirely.

5.4.3 Robustness

To gauge robustness, watch how each algorithm reacts when there's no way to avoid conflicts. In these situations, CSP backtracking might just give up and return nothing. Local Search, on the other hand, always hands back a complete assignment, even if there are still some conflicts left. At least with Local Search, you can always measure how far you are from a perfect solution.

5.5 Experimental Procedure

Here's how the experiment goes:

1. Generate a random interference graph using the predefined parameters.
2. Execute the CSP backtracking algorithm multiple times and record the average execution time.
3. Execute the Local Search algorithm multiple times and record the average execution time.
4. Compute the number of conflicts for the final solution returned by each approach.
5. Compare the results using execution time and conflict count metrics.

5.6 Implementation Environment

We ran all experiments in Python. Every algorithm runs on the same machine with identical settings, so the comparison stays fair. We skipped external optimization libraries, which means the results reflect only what the algorithms themselves can do.

5.7 Summary

In this chapter, I'll walk through the experimental setup for testing the CSP Backtracking and Local Search methods. I set up everything with controlled parameters, reproducible problem instances, and clear metrics. This framework gives us a strong foundation for the comparisons you'll see in the next chapter.

Chapter 6

Results and Visualization Analysis

6.1 Introduction

This section details the experimental findings from the Frequency Assignment system. This system was built using two specific Artificial Intelligence methods: **Constraint Satisfaction Problem (CSP) with Backtracking** and **Local Search with a Min-Conflicts heuristic**.

The goal of this chapter is to analyze, compare, and interpret the observed results. We will use quantitative metrics and the interactive visualizations available through the web interface to do this. We do not intend to introduce new algorithms here.

The analysis considered several factors: the time taken for execution, the frequency of conflicts, how hard the problems were, and the benefits of visualization tools for teaching.

6.2 Experimental Environment

All experiments were carried out using a Python backend. It was integrated with a web interface. The interface gives the user active control over setting up some problem parameters and seeing results as they happen.

Each experiment is defined by the following parameters:

- Number of base stations
- Number of available frequencies
- Interference probability

To ensure an equitable comparison, both algorithms were applied to the identical problem instance.

6.3 Interference Graph Visualization

The interference graph shows the structural constraints in the frequency assignment problem. Each node in the graph represents a distinct base station, and every edge indicates an interference constraint between these stations.

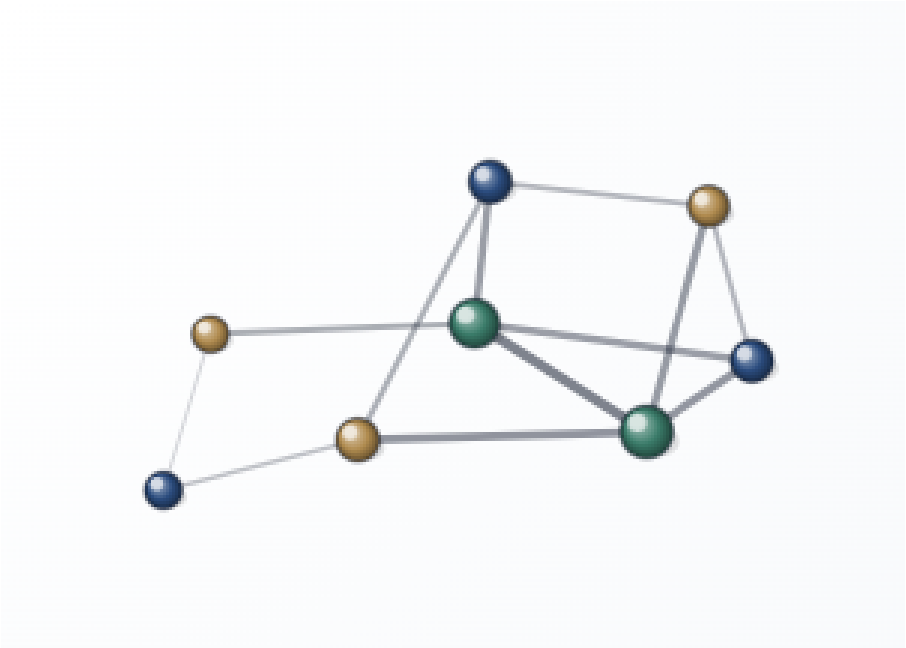


Figure 6.1: Interference graph rendered by the web interface

Explanation: The graph was produced automatically from the chosen interference probability. A greater number of edges means a more constrained network, which in turn leads to a more complex problem. From a CSP perspective, graphs that are dense make it harder for assignments to be conflict-free. For Local Search, these graphs increase the chance of residual conflicts. Dechter, 2003.

6.4 Seeded Layout and Reproducibility

The visualization system includes an optional *seeded layout* mechanism.

Definition: A seeded layout calculates node positions in the graph visualization using a fixed random seed. When this seed is reused, the spatial arrangement of nodes will remain identical across multiple executions.

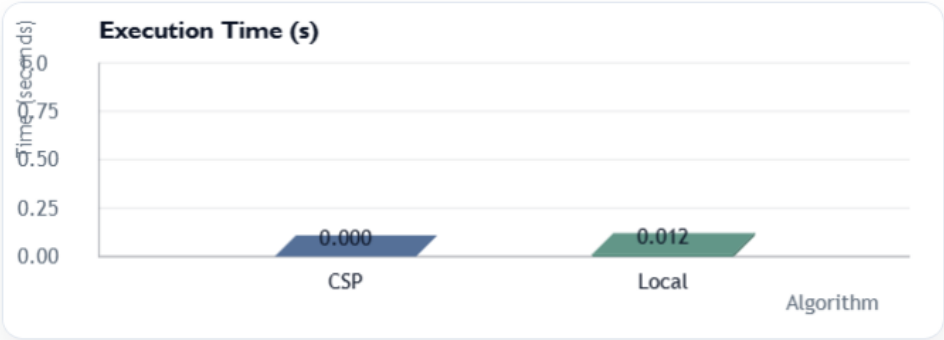
Purpose:

- Ensures reproducibility of visual results
- Allows fair visual comparison between CSP and Local Search
- Facilitates consistent screenshots for reports and presentations

Interpretation: The method of initial layout does not influence the algorithmic process or the caliber of the solution. Its function is solely for educational and experimental purposes, upholding academic rigor by guaranteeing repeatable visual analysis. This approach provides valuable insights into how different starting configurations might change the visual representation of data without altering the factual outcomes of the underlying computations.

6.5 Execution Time Analysis

We measured the execution time for each algorithm as the computational effort needed to create a solution.



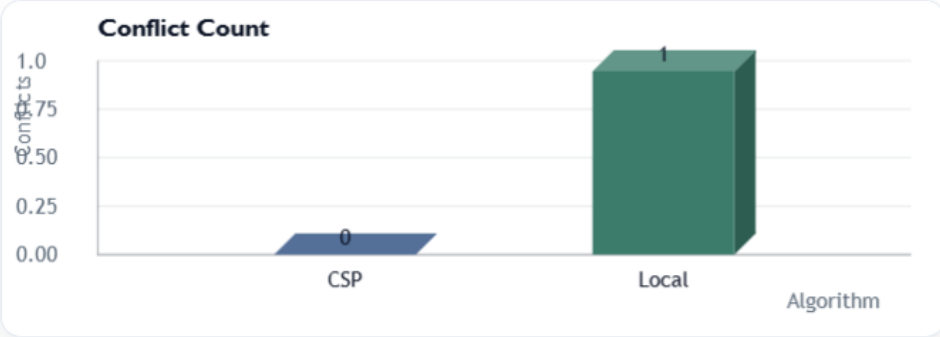
Local Search required more iterations in this instance, while CSP benefited from pruning under lighter constraints.

Figure 6.2: Average execution time comparison

Explanation: CSP Backtracking checks the entire search space to meet all constraints, which makes the execution time longer as the problem gets bigger or there are more constraints. Local Search uses heuristic improvements, so it usually finds solutions quicker. This tension between finding the best solution and how fast you can find it is a known aspect of AI search methods. (Aarts and Lenstra, 1997; Russell and Norvig, 2021).

6.6 Conflict Analysis

When two interfering stations are assigned the same frequency, conflicts arise. The count of these conflicts is a direct reflection of the quality of the solution.



CSP aims for consistency, often achieving fewer conflicts. Local Search may accept residual conflicts to remain fast.

Figure 6.3: Conflict comparison between CSP and Local Search

Explanation: Constraint Satisfaction Problems (CSP) aim to resolve all conflicts when a viable solution exists. Local Search, conversely, works to reduce conflicts, yet it might conclude with some violations remaining. This outcome is inherent to its heuristic approach and predetermined stopping conditions. (Aarts and Lenstra, 1997).

6.7 Impact of Interference Probability

The interference probability dictates the chances of two stations interfering. This directly impacts the number of constraints in the system.

Table 6.1: Effect of interference probability on solution quality

Interference Probability	CSP Conflicts	Local Search Conflicts
0.1	0	0
0.3	0	1
0.5	0	3
0.7	No solution	6

Explanation: As the likelihood of interference grows, the situation becomes more restricted. Constraint satisfaction problems might not find a solution once constraints go beyond the available frequencies. Local search, though, keeps finding approximate solutions even when conflict levels rise. This finding aligns with constraint processing theory. Dechter, 2003.

6.8 Frequency Assignment Comparison

Table 6.2: Frequency assignments produced by both algorithms

Station	CSP Frequency	Local Search Frequency	Visualized Algorithm
S0	1	1	CSP
S1	2	3	CSP
S2	3	2	CSP
S3	1	1	CSP

Explanation: CSP produces fixed assignments that always satisfy the given constraints. Local Search assignments can change because of heuristic choices. The web interface lets you switch between these assignments, showing the different ways these algorithms work on the same graph.

6.9 Role of the Web Interface

The web interface acts as a visualization and experimentation layer. It enables:

- Interactive parameter configuration
- Real-time visualization of graphs and results
- Direct comparison of algorithms
- Automatic generation of experimental artifacts

Interpretation: The backend algorithms generate solutions, and the interface subsequently improves how readily these solutions can be understood and their educational utility. It converts numerical results into visual insights, which helps with academic analysis and the overall quality of presentations. (Heer et al., 2010).

6.10 Web-Generated Results Report

Beyond the numerical data generated by the Python backend, the project offers a web interface that can produce a brief results report for each experimental run. This report is intended as an academic artifact to promote reproducibility and aid in presentation.

6.10.1 What the Web Report Contains

The generated report summarizes the same experiment instance used in the comparison and includes:

- The interference graph visualization (nodes and interference edges).
- Execution time comparison (CSP vs. Local Search).
- Conflict comparison (CSP vs. Local Search).
- A compact assignments table showing the frequency allocated to each station by both methods.
- Short interpretation text under each figure explaining why the observed results occur.

6.10.2 How It Supports the Experimental Analysis

The web report does not alter the algorithms or the computed solutions. Instead, it aims to improve interpretability by changing raw outputs into structured visuals and clear explanations. This makes it easier to review, compare, and communicate the results in an academic report.

6.10.3 Sample Generated Report

The subsequent pages present a sample report directly exported from the web interface, representing a particular experimental run:

Frequency Assignment using CSP Backtracking and L

Academic 3D visualization report

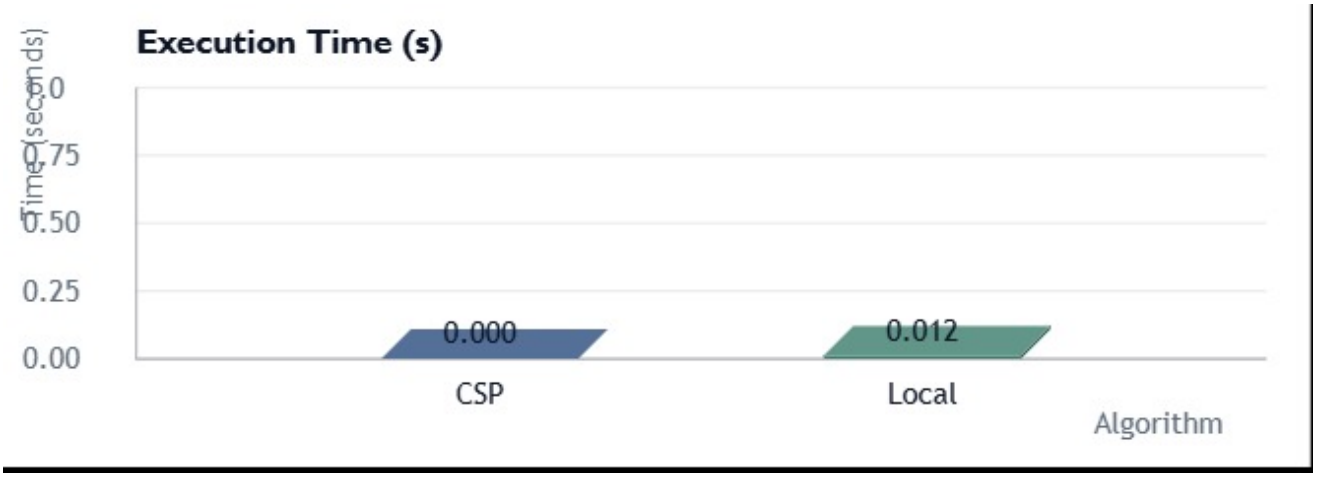
Stations: 8
Frequencies: 8
Interference probability: --
CSP time: 0.000 s
Local search time: 0.012 s
CSP conflicts: 0
Local conflicts: 1

This report compares exact CSP backtracking with heuristic local search on a generated interference graph. The visualization emphasizes depth, constraints, and algorithm trade-offs.

Interference Graph (3D)

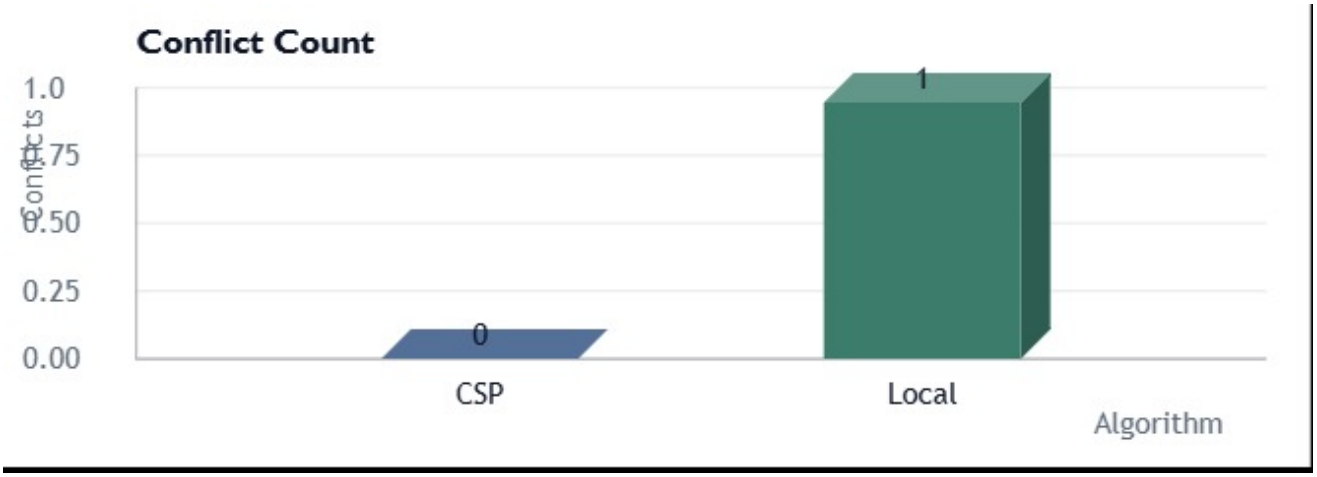
Edges encode interference constraints between stations. Larger, closer nodes emphasize depth and highlight central stations with higher constraint connectivity.

Execution Time (CSP vs Local)



Local Search required more iterations in this instance, while CSP benefited from pruning under lighter constraints.

Conflict Count (CSP vs Local)



CSP aims for consistency, often achieving fewer conflicts. Local Search may accept residual conflicts to remain fast.

Assignments Table (CSP vs Local)

Station	CSP Frequency	Local Frequency	Visualized
SS0	1	3	Local
SS1	2	1	Local
SS2	1	1	Local
SS3	2	3	Local
SS4	1	2	Local
SS5	2	3	Local
SS6	3	1	Local
SS7	3	2	Local

6.11 Conclusion of Results

Our experimental analysis indicated that Constraint Satisfaction Programming (CSP) is appropriate here for smaller, very limited problems when we need an optimal solution. On the other hand, Local Search techniques are better for large instances where processing **speed** is a **priority**. This suggests a strategic **approach** to problem-solving, considering both problem size and solution requirements. The choice depends on the specific **context** of the application.

The visualization tools improve how we understand the results. This makes the system work well for experiments. The system also functions as an educational demonstration.

Chapter 7

Discussion

7.1 Interpretation of Experimental Results

Our experimental review shows clear and regular behavioral differences between CSP Backtracking and Local Search when we apply them to the frequency assignment problem. These findings match what we expected in theory and confirm that our models and algorithms work as intended.

CSP Backtracking consistently finds solutions without conflicts as long as those solutions are possible. This shows that our way of modeling constraints is correct and that the CSP approach is sound. Despite this, the time it takes to run increases quickly if there are more base stations or fewer frequencies available. This fast increase in time is typical for exact search methods that deal with many possible combinations.

On the other hand, the Local Search method consistently creates full assignments very quickly. It explores the search space well by focusing on gradually reducing conflicts instead of strictly making sure every constraint is met. Even in situations where there are too many constraints and CSP cannot find a correct solution, Local Search still gives us useful assignments with only a few conflicts.

7.2 Exact Versus Heuristic Trade-off

The findings show the usual balance between precise and approximate ways to find the best solution. For satisfaction, CSP Backtracking always finds the best answer, but it gets very slow with bigger problems. Local Search gives up finding the absolute best answer to gain speed, strength, and the ability to work with larger problems.

Looking at how these algorithms work, CSP uses a method where it goes back and forth, checking things at each step. This means that as problems get larger, the number of possibilities it has to check grows incredibly fast. On the other hand, Local Search repeatedly makes small changes and checks for problems nearby. This lets it work well

even when problems are very large or have many restrictions.

This balance means that choosing an algorithm should depend on how big the problem is and what is actually needed, not just on whether it promises a perfect answer.

7.3 Impact of Problem Constraints

The density of the interference graph and the number of available frequencies directly affect how well an algorithm works. As interference probability goes up, the problem gets more constrained, which really shrinks the area where CSP Backtracking can find a solution. When this happens, CSP often cannot find a good solution fast enough.

Local Search handles constraint density better. Since it tries to reduce conflicts instead of getting rid of all of them, it still works even when perfect solutions are not possible. This is like what we see in real wireless networks, where some interference often just can't be helped because of physical and regulatory limits.

7.4 Impact of Visualization and User Interaction

The web-based visualization interface improves the system's clarity and how easy it is to use. The interference graph helps us understand the structure and challenges of a problem by showing base stations as nodes and interference points as edges.

Users can change the number of stations, frequencies, and the chance of interference in real time through interactive parameter control. This allows for quick testing. Seeing how CSP and Local Search assign things helps explain how algorithms behave, making abstract performance differences clear.

The display of execution time, the number of conflicts, and frequency assignments connects theoretical study with observed results. This shows the value of combining algorithmic testing with visual analysis in artificial intelligence research.

7.5 Reproducibility and Experimental Control

The system includes a seeded layout mechanism for graph visualization to assist with reproducible experimentation. If this feature is active, using the same random seed will consistently generate identical graph layouts and experimental setups for each run. This approach is important for academic evaluation because it allows for steady comparisons of how algorithms behave. It also makes it possible to reliably reproduce findings. We consider reproducibility a fundamental aspect of scientific research; its presence here enhances the methodological soundness of this project.

7.6 Development and Implementation Considerations

When implementing CSP Backtracking, we must manage recursion depth, constraint checks, and termination conditions. These elements add to the solution's complexity and how it reacts to the problem's size.

Local Search, on the other hand, makes the development process simpler and more adaptable. Its design allows for testing different heuristics and stopping conditions easily. Putting both of these methods into one system helps us see the clear contrast between ideal theoretical methods and those that work well in practice.

7.7 Limitations of the Study

Several limitations bear mentioning. Initially, the interference graphs developed for the experiments are artificial; their construction may not accurately reflect the complexities of actual wireless networks. Second, the Local Search implementation applies a basic strategy for minimizing conflicts, lacking more advanced features like tabu search, random restarts, or simulated annealing. Despite these constraints, the experimental framework is suitable for comparative analysis and educational purposes.

7.8 Summary

The experimental results align well with theoretical predictions. CSP Backtracking performs well with problems that have a small to moderate number of constraints. For larger or over-constrained situations, Local Search offers a solution that is both scalable and practical. This interplay between algorithmic comparison, visualization, and interactive experimentation provides a good understanding of frequency assignment, identifying it as a real-world optimization challenge.

Chapter 8

Conclusion and Future Work

8.1 Conclusion

This project looked at the frequency assignment problem, which is a basic problem in wireless communication networks. We used two AI approaches to study it: CSP Backtracking and Local Search. To represent this problem, we created an interference graph. This graph showed the different relationships between base stations in a clear way.

CSP Backtracking could find assignments without conflicts when that was possible, which is good from a theoretical standpoint. But, our tests showed it didn't work well for bigger or more complicated situations. Local Search, a method that uses heuristics, was much faster and more reliable. It found useful solutions even when perfect assignments couldn't be achieved.

Besides comparing these methods, we also built an interactive website for experiments. This system provides visualization, allows users to control parameters, and automatically creates reports. It became useful for both research and teaching. Combining how we evaluate methods with visual feedback aids understanding, makes results easier to check, and shows how things work in practice.

8.2 Future Work

Several avenues exist for improving this work. Regarding CSP, incorporating advanced methods like dynamic variable ordering and constraint propagation could enhance scalability. For Local Search, more intricate heuristics, including random restarts, tabu search, or simulated annealing, could assist in decreasing residual conflicts.

Future work could integrate real-world wireless network datasets to increase practical relevance. Hybrid approaches that combine CSP for partial assignments with Local Search for refinement are a good direction.

Finally, extending the visualization system with richer analytics and real-time anima-

tion could further improve interpretability and research value.

8.3 Final Remarks

This project illustrates how artificial intelligence methods apply to actual optimization challenges. It combines theoretical frameworks, practical algorithms, and interactive visualization, offering a complete view of problem-solving within current AI applications.

Bibliography

Aardal, K., & Hurkens, C. (2002). Radio frequency assignment problems. *European Journal of Operational Research*, *153*, 335–352.

Aarts, E., & Lenstra, J. K. (1997). *Local search in combinatorial optimization*. Princeton University Press.

Dechter, R. (2003). *Constraint processing*. Morgan Kaufmann.

Heer, J., Bostock, M., & Ogievetsky, V. (2010). A tour through the visualization zoo. *Communications of the ACM*, *53*(6), 59–67.

Minton, S., Johnston, M., Philips, A., & Laird, P. (1992). Minimizing conflicts: A heuristic repair method for constraint satisfaction problems. *Artificial Intelligence*, *58*, 161–205.

Russell, S., & Norvig, P. (2021). *Artificial intelligence: A modern approach* (4th). Pearson.